

Question 1: Heuristic design and admissibility [15 points]

For each of the three cost measures—steps, Scrabble, and frequency—describe the heuristic implemented in your program.

For each heuristic:

1. Give a precise formula or pseudocode.
2. Explain what unavoidable remaining cost it lower-bounds.
3. Explain why it cannot overestimate the true optimal remaining cost.

Your explanation must be specific to the corresponding cost measure. Merely stating that the heuristic “is a lower bound” is insufficient.

1. `mismatches = sum(1 for a, b in zip(node.state, self.goal) if a != b)`

If `self.cost == 'steps':`

`return mismatches`

This is the lowest value the heuristic can be, because it needs to have enough steps to at least change the letters that do not match the goal word, since each step can only have 1 action of changing a single letter in the word.

It cannot overestimate the corresponding cost measure because the cost must be at least the number of different letters between initial word and goal word, no less than this.

`mismatches = sum(1 for a, b in zip(node.state, self.goal) if a != b)`

If `self.cost == 'scrabble':`

`return mismatches * 1`

This is the lowest value the heuristic can be, because it needs to have enough steps to at least change the letters that do not match the goal word, since each step can only have 1 action of changing a single letter in the word, and the lowest cost of a letter in scrabble is 1.

It cannot overestimate the corresponding cost measure because the cost must be at least the number of different letters between the initial word and goal word multiplied by the lowest scrabble value, 1, and no less than this.

`mismatches = sum(1 for a, b in zip(node.state, self.goal) if a != b)`

If `self.cost == 'frequency':`

`min_cost = min(dictionary.values())`

`return (dictionary[self.goal] + (mismatches - 1) * min_cost)`

This is the lowest value the heuristic can be, because the cost must be at least that of the rarity of the goal word, and every additional step will be at least the number of

mismatches (minus 1, the goal word), which will cost as much as the lowest rarity, therefore giving us the lowest value possible.

It cannot overestimate the corresponding cost measure because the cost can be no lower than the rarity of the goal word and the amount of the mismatches minus the goal word multiplied by the minimum rarity available, since there will have to be additional steps for each letter that is different from the initial word.

Question 2: Does the heuristic actually help? [10 points]

Run both uniform-cost search and A* on:

- WHY → ASK
- BEAR → DUCK

Run each problem using all three cost measures. Report:

Word Pair	Cost measure	UCS cost	UCS nodes expanded	A* cost	A* nodes expanded	Reduction
WHY -> ASK	Steps	5	773	5	16	97%
WHY -> ASK	Scrabble	9	765	9	353	53%
WHY -> ASK	Frequency	38.85	808	38.855	807	0.12%
BEAR -> DUCK	Steps	4	691	4	6	99%
BEAR -> DUCK	Scrabble	11	2981	11	1376	53%
BEAR -> DUCK	Frequency	38.24	1084	38.24	1078	0.55%

Calculate the percentage reduction as:

$$100 \times ((\text{UCS expansions} - \text{A* expansions}) / \text{UCS expansions})$$

A negative result means that A* expanded more nodes than UCS.

Then answer:

1. Did UCS and A* always return the same optimal cost? If not, identify and investigate the discrepancy.
Yes, they always returned the same optimal cost.
2. For each word pair, under which cost measure did A* produce the largest percentage reduction in expanded nodes?

For why & ask, the largest percentage reduction in expanded nodes was with the cost measure steps.

For bear & duck, the largest percentage reduction in expanded nodes was with the cost measure steps

3. For each word pair, under which cost measure was the heuristic least informative?
For each word pair, under Frequency the heuristic was the least informative.

4. Explain the differences using the definitions of your heuristics, not runtime alone.

For steps, there was a high percentage of reduction because the heuristic formula evaluates letter differences, and every step has a cost of 1. Since it is a direct estimate of the minimum steps needed, A* focuses its search towards the goal, making the reduction vast in both cases

For scrabble, the heuristic formula assumed that the letter being changed had a value of 1. However, in the worst case letters can be valued at 10. Therefore, the heuristic formula undervalues the real cost, which makes it less effective.

For frequency, there was a very low percentage of reduction because the heuristic formula has a very low estimate, making it further from the actual cost, which makes the A* search less effective.

Question 3: Reversibility and direction-dependent costs [5 points]

DOGCAT actions are reversible: if one legal word can be changed into another, the reverse change is also legal.

For each cost measure—steps, Scrabble, and frequency—determine whether the optimal cost from W1 to W2 must always equal the optimal cost from W2 to W1.

For each cost measure:

1. State yes or no.
2. Justify your conclusion.
3. If the answer is no, give a concrete counterexample using words from the provided dictionary. Clearly distinguish between the reversibility of an action and the symmetry of its cost.

Steps:

Yes, the optimal cost from W1 to W2 must always equal the optimal cost from W2 to W1 because the only thing that increases the cost is the amount of steps taken, and you can use the same words to get to W1 and W2.

Scrabble:

No, the optimal cost from W1 to W2 will not always equal the optimal cost from W2 to W1. This is because the cost of the letters varies, and if W2 has higher cost letters than W1,

then the cost of W2 to W1 will be lower than W1 to W2. For example, shop to glob has a cost of 6, while glob to shop has a cost of 5.

Frequency:

No, the optimal cost from W1 to W2 will not always equal the optimal cost from W2 to W1. This is because the rarity of the words differ, and so if W1 has a higher rarity than W2, then W1 to W2 will have a lower cost than W2 to W1. For example, shop to glob has a cost of 33.53 while glob to shop has a cost of 25.53.

Question 4: LLM heuristic audit [10 points]

Give a generative AI system the following prompt:

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Submit:

1. The model used, the exact prompt, and its proposed heuristic. A complete conversation transcript is not required.

Model used: GPT5.6 Luna

In DOGCAT, the initial and goal states are lowercase words of the same length—three or four letters—and every legal state must appear in the supplied dictionary. A legal action changes exactly one letter. Under the frequency objective, entering word w costs $1 + \text{rarity}(w)$, where the nonnegative rarity value is stored in the dictionary. The baseline heuristic $h_0(n)$ is the number of positions in which state n differs from the goal. Propose a rarity-aware heuristic intended to be more informative than h_0 while remaining admissible. Define it precisely and justify your proposal. By “more informative,” we mean that the proposed heuristic should be at least as large as h_0 for every state and strictly larger for at least one state, without overestimating the true optimal remaining cost.

Show more ▾

Worked for 17s >

A simple rarity-aware heuristic that is guaranteed to be admissible is to account for the **rarity of the goal word**, since every solution that reaches the goal must eventually enter the goal.

Let

$$d(n) = h_0(n)$$

be the number of positions in which n differs from the goal, and let $r(g)$ be the rarity of the goal word g .

Define:

$$h_R(n) = \begin{cases} 0, & n = g \\ h_0(n) + r(g), & n \neq g \end{cases}$$

+ Ask anything

⌂ Think 🔊

If initial word = goal word, heuristic is 0. Otherwise, the heuristic is equal to the number of positions where initial word differs from goal word plus the rarity of the goal word.

2. A precise formula or pseudocode for the proposed heuristic.

Final proposed heuristic

$$h_R(n) = \begin{cases} 0 & \text{if } n = g, \\ \#\{\text{positions where } n \text{ differs from } g\} + \text{rarity}(g) & \text{otherwise.} \end{cases}$$

If the particular DOGCAT instance has $r(g) = 0$, then this heuristic collapses to h_0 ; in that case, **no strict improvement can be guaranteed from the goal's rarity alone.**

3. Determine whether the heuristic is admissible. Give either a general proof or a concrete counterexample.

The heuristic is admissible because it must be at least the rarity of the goal word and how many differences there are between the characters in initial word versus goal word. This is because the lowest the heuristic can be without being 0 (if the initial word and the goal word are the same) is the rarity of the goal word + 1 (as outlined in the document provided to us).

Let $N \neq G$ and suppose N differs from the goal in D positions

Because any legal action only allows one letter to be changed, any path from N to G requires at least D actions. Each action adds a word W , which costs $1 + \text{rarity}(W)$

The action must also find the goal word, G , and that action costs $1 + \text{rarity}(G)$

The other $D - 1$ actions each cost at least 1, therefore every path from N to G costs at least $(D - 1)(1) + (1 + \text{rarity}(G)) = D + \text{rarity}(G)$

Since $h_R(n) = D + \text{rarity}(G)$

Then $\overline{h_R(n)} \leq h^*(n)$, where $h^*(N)$ is the true optimal cost, therefore this heuristic is admissible for every positive rarity dictionary

4. If the proposal is invalid, make the smallest repair you can or replace it with a safe alternative.
N/A

5. Test the verified or repaired heuristic against h_0 on the two frequency-cost problems from Question 2. Compare solution costs and numbers of expanded nodes, then conclude whether the evidence supports using the proposed heuristic.

why -> ask

UCS:

Frequency solution cost: 38.85

Number of expanded nodes: 808

A*:

Frequency solution cost: 38.85

Number of expanded nodes: 755

A 6.55% reduction, much greater than the reduction in question 2. Supports using proposed heuristic due to higher reduction

bear -> duck

UCS:

Frequency solution cost: 38.24

Number of expanded nodes: 1084

A*:

Frequency solution cost: 38.24

Number of expanded nodes: 720

A 33.57% reduction, MUCH greater than the reduction in question 2, which supports using the proposed heuristic.